Rockchip RK809 Developer Guide

ID: RK-KF-YF-058

Release Version: V1.0.0

Release Date: 2019-11-26

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2019. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily introduces the various sub-modules of the RK809, covering related concepts, functionalities, DTS configuration, and analysis and troubleshooting of common issues.

Product Version

Chip Models	Kernel Version
RK809	4.4, 4.19

Intended Audience

This document (this guide) is mainly intended for:

- Technical Support Engineer
- Software Development Engineer

Revision History

Date	Version	Author	Description
2019.11.26	V1.0.0	Zhang Qing	Initial version release

Rockchip RK809 Developer Guide

1. Basics
    - 1.1 Overview
    - 1.2 Functions
    - 1.3 Chip Pin Functions
    - 1.4 Key Concepts
    - 1.5 Power-On Conditions and Sequence
2. Configuration
    - 2.1 Driver and menuconfig
    - 2.2 DTS Configuration
    - 2.3 Function Interfaces
3. Debug
    - 3.1 4.4 Kernel
    - 3.2 4.19 Kernel

1. Basics

1.1 Overview

The RK809 is a high-performance PMIC integrating five high-current DCDC converters, nine LDOs, two switches, one RTC, one high-performance CODEC, and configurable power-on sequencing.

Power rails in the system are generally categorized into two types: DCDC and LDO. Their general characteristics are as follows (refer to external resources for detailed specifications):

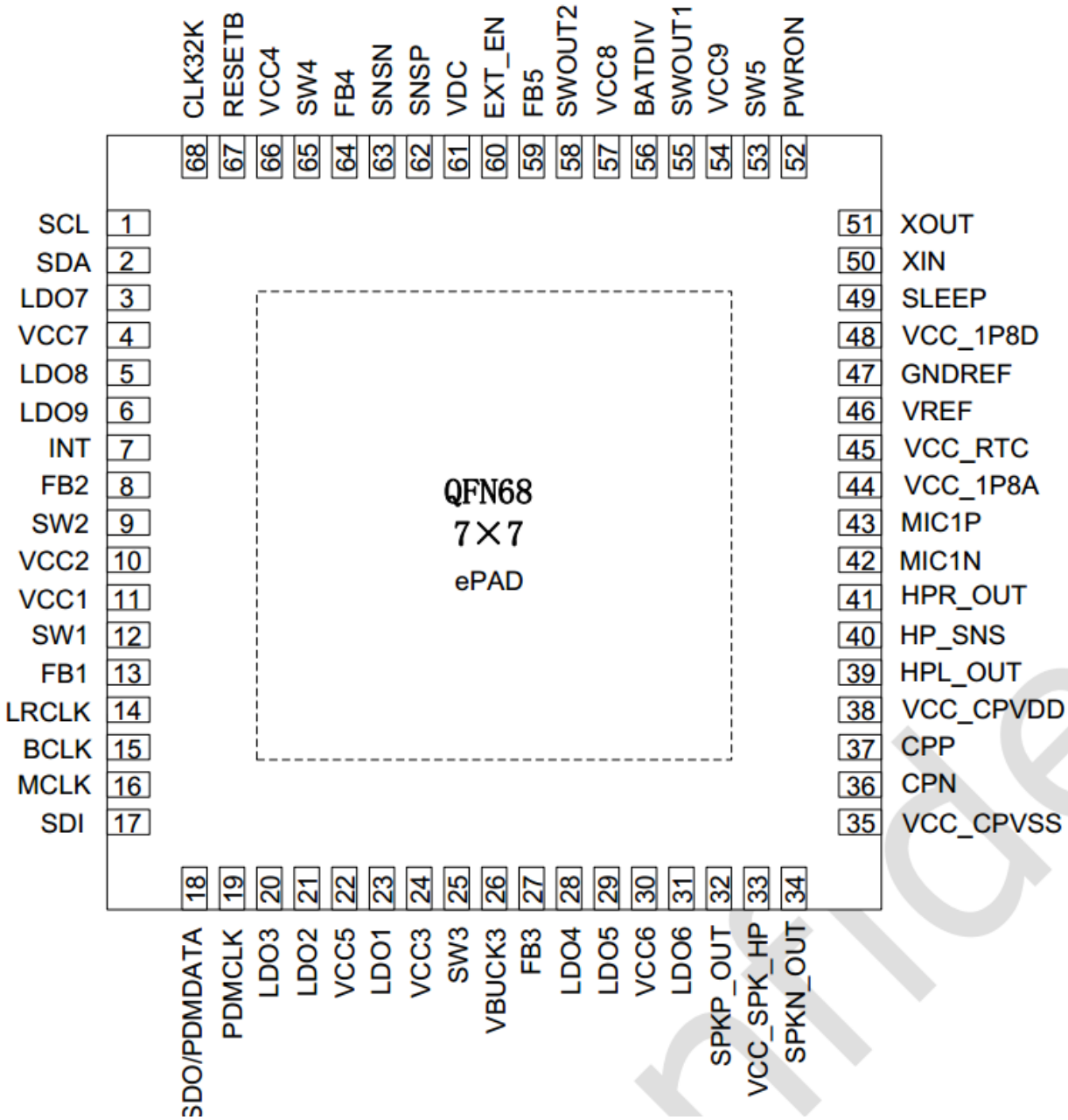
1. DCDC: Offers high efficiency when the input-output voltage differential is large, but exhibits relatively high ripple and higher cost. Therefore, it is used for applications with large voltage differentials and high current loads. It typically operates in two modes: PWM mode provides good ripple and transient response but lower efficiency; PFM mode offers higher efficiency but poorer load capability.
2. LDO: Exhibits low efficiency and low cost when the input-output voltage differential is large. To improve LDO conversion efficiency, system-level optimizations are applied; for example, for an LDO outputting 1.1V, its input can be supplied by a DCDC rail such as VCCIO_3.3V to enhance efficiency. Therefore, wherever the circuit allows, LDOs should be connected to DCDC output rails, while paying attention to power-on sequencing.

1.2 Functions

From the user's perspective, the functions of the RK809 can be summarized into six parts:

1. Regulator function: Controls the power status of various DCDC and LDO rails.
2. RTC function: Provides clock timing and alarm functions.
3. GPIO function: Can be used as standard GPIOs with pinctrl capabilities.
4. Pwrkey function: Detects power button press/release events, saving one GPIO for the AP.
5. CLK function: Provides two 32.768kHz clock outputs; one is always-on and non-configurable, while the other is software-controllable.
6. Codec function: Supports sampling rates up to 192kHz, 16-bit and 32-bit data formats, and interfaces including DAC, ADC, and PDM. (This function is not covered in this document and will be detailed in a dedicated document later.)

1.3 Chip Pin Functions



In the following description, special attention should be paid to the SLEEP and INT pins, particularly the SLEEP pin which supports extended GPIO functionality:

PIN NO	PIN NAME	PIN DESCRIPTION
1	SCL	I2C clock input
2	SDA	I2C data input and output
3	LDO7	LDO7 output
4	VCC7	Power supply of LDO7/8/9
5	LDO8	LDO8 output
6	LDO9	LDO9 output
7	INT	Interrupt request pin, open drain
8	FB2	Output feedback voltage of buck2
9	SW2	Switching node of buck2
10	VCC2	Power supply of buck2
11	VCC1	Power supply of buck1
12	SW1	Switching node of buck1
13	FB1	Output feedback voltage of buck1
14	LRCLK	The I2S framing clock
15	BCLK	The I2S bit clock
16	MCLK	The I2S main clock input pin
17	SDI	The I2S DAC input data
18	SDO/PDMDATA	The I2S ADC output data/PDM Data for the DSADC
19	PDMCLK	PDM CLK for the DSADC OUTPUT
20	LDO3	LDO3 output
21	LDO2	LDO2 output
22	VCC5	Power supply of LDO1/2/3
23	LDO1	LDO1 output

Rockchip

PIN NO	PIN NAME	PIN DESCRIPTION
24	VCC3	Power supply of buck3
25	SW3	Switching node of buck3
26	VBUCK3	Output voltage of buck3
27	FB3	Output feedback voltage of buck3
28	LDO4	LDO4 output
29	LDO5	LDO5 output
30	VCC6	Power supply of LDO4/5/6
31	LDO6	LDO6 output
32	SPKP_OUT	Positive speaker driver output
33	VCC_SPK_HP	Power supply for speaker and charger pump
34	SPKN_OUT	Negative speaker driver output.
35	VCC_CPVSS	Negative power supply for the headphone
36	CPN	Negative switching node of the charger pump
37	CPP	Positive switching node of the charger pump.
38	VCC_CPVDD	Positive power supply for the headphone
39	HPL_OUT	Left channel output of the headphone
40	HP_SNS	Reference ground for the headphone
41	HPR_OUT	Right channel output of the headphone
42	MICIN	Negative input of the Microphone
43	MICIP	Positive input of the Microphone
44	VCC_1P8A	Power supply for internal 1.8V analog circuit
45	VCC_RTC	Power supply filter
46	VREF	Internal reference voltage
47	GNDREF	Reference ground
48	VCC_1P8D	Power supply for internal 1.8V digital circuit

Confidential

49	SLEEP	Sleep mode control input
50	XIN	32.768KHz crystal oscillator input
51	XOUT	32.768KHz crystal oscillator output
52	PWRON	Power on key input, active low, internal 17k resistor pull high to VCC_RTC
53	SW5	Switching node of BUCK5
54	VCC9	Power supply of buck5 and SWOUT1
55	SWOUT1	Power switch out 1
56	BATDIV	Divided voltage of positive battery
57	VCC8	Power supply of SWOUT2
58	SWOUT2	Power switch out 2
59	FB5	Output feedback voltage of buck5
60	EXT_EN	Enable Signal for external high voltage BUCK
61	VDC	If it exceeds 0.55V for the first time, it will start the PMIC(rising edge triggering start).And it is connected to the divider of external power supply generally.
62	SNSP	Bat charging and discharging sense current positive pin
63	SNSN	Bat charging and discharging sense current negative pin
64	FB4	Output feedback voltage of buck4
65	SW4	Switching node of buck4
66	VCC4	Power supply of buck4
67	RESETB	Reset pin after power on, active low
68	CLK32K	32.768KHz clock output, open drain
Exposed pad	Exposed ground	Ground

Rockchip

1.4 Key Concepts

- I2C Address

7-bit slave address: 0x20

- The PMIC has three operating modes

1. PMIC Normal Mode

When the system is running normally, the PMIC operates in normal mode, with the pmic_sleep pin held low.

2. PMIC Sleep Mode

To minimize standby power consumption during system suspend, the PMIC switches to sleep mode. In this mode, certain output voltages may be reduced or outputs disabled entirely, configurable according to specific product requirements. During system standby, the AP configures the pmic_sleep mode via an I2C command and then drives the pmic_sleep pin high to enter sleep state. Upon SoC wakeup, the pmic_sleep pin returns to low, causing the PMIC to exit sleep mode.

3. PMIC Shutdown Mode

When the system initiates a shutdown sequence, the PMIC must power down the entire system. The AP configures the pmic_sleep mode to shutdown via an I2C command and then drives the pmic_sleep pin high to enter shutdown state.

- pmic_sleep Pin

This pin is normally low, keeping the PMIC in normal mode. Driving the pin high switches the PMIC to either sleep or shutdown mode. On the RK809, this pin supports multiplexing functions configurable via pinctrl:

1. SLEEP function: Used for switching to sleep mode;
2. Shutdown function: Used for POWER DOWN;
3. Reset function: Used for RESET;
4. Idle: No function assigned;

- pmic_int Pin

This pin is normally high. It transitions to low when an Interrupt occurs. If the Interrupt remains unhandled, the pin stays low.

- pmic_pwron Pin

The pwrkey function requires the hardware power button to be connected to this pin. The Driver uses this pin to detect button press and release events.

- Operating Modes of Individual DCDC Rails

  DCDC supports PWM (also known as force PWM) and PFM modes. However, the PMIC features a mode that dynamically switches between PWM and PFM, commonly referred to as AUTO mode. The PMIC supports two configurations: PWM mode and AUTO PWM/PFM mode. While AUTO mode offers higher efficiency, it exhibits poorer ripple and transient response. For system stability, the device is configured to operate in PWM mode during runtime and switches to AUTO PWM/PFM mode when the system enters sleep state.

- DCDC3 Voltage Regulation

The DCDC3 power rail is unique; its voltage cannot be modified via registers and must be adjusted using external voltage divider resistors. Therefore, to change the voltage, the peripheral hardware must be modified. In Rockchip solutions, this rail is typically used as VCC_DDR.

- Runtime Voltage Adjustment Range for DCDC and LDO

1. Discontinuous DCDC Voltage Range:

Voltage Range (V)	Step Size (mV)	Specific Voltage Levels (V)
0.7125 ~ 1.5	12.5	0.7125, 0.725, 0.7375, ..., 1.5
1.6 ~ 2.4	100	1.6, 1.7, 1.8, 1.9, ..., 2.4

2. Continuous LDO Voltage Range:

Voltage Range (V)	Step Size (mV)	Specific Voltage Levels (V)
0.6 ~ 3.4	25	0.6, 0.625, 0.65, 0.675, ..., 3.4

1.5 Power-On Conditions and Sequence

1. Power-On Conditions

The PMIC can be powered on if any of the following conditions are met:

- The EN signal transitions from low to high.
- The EN signal remains high, and an RTC alarm Interrupt is triggered.
- The EN signal remains high, and the PWRON button is pressed.
- The EN signal remains high, and a charger is inserted.

2. Power-On Sequence

Power-on sequence requirements for each power rail may vary across different SoC platforms. The current power-on sequences are listed below; please refer to the latest datasheet for specific details:

			RK809-1	
	Range of output voltage	**Maximum output current**	**Default voltage**	**Start up sequence**
BUCK1	0.5V-2.4V	2.5A	1.0V	2
BUCK2	0.5V-2.4V	2.5A	1.0V	2
BUCK3	X(external divided resistor) Or 0.5V-2.4v(internal divided resistor)	1.5A	x	4
BUCK4	0.5V-3.4V	1.5A	3.0V	5
LDO1	0.6V-3.4V	400mA	2.5V	3

			RK809-1	
	Range of output voltage	**Maximum output current**	**Default voltage**	**Start up sequence**
LDO2	0.6V-3.4V	400mA	1.8V	3
LDO3	0.6V-3.4V	400mA	1.0V	2
LDO4	0.6V-3.4V	100mA	3.0V	5
LDO5	0.6V-3.4V	400mA	3.0V	5
LDO6	0.6V-3.4V	400mA	3.0V	5
LDO7	0.6V-3.4V	400mA	2.8V	OFF
LDO8	0.6V-3.4V	400mA	1.8V	OFF
LDO9	0.6V-3.4V	400mA	1.5V	OFF
BUCK5	1.5V-3.6V	2.5A	3.3V	1
SWOUT1				OFF
SWOUT2				OFF

2. Configuration

2.1 Driver and menuconfig

4.4 Kernel Configuration

RK809 driver files (partially shared with rk817 and rk805):

```
drivers/mfd/rk808.c
drivers/input/misc/rk8xx-pwrkey.c
drivers/rtc/rtc-rk808.c
drivers/gpio/gpio-rk8xx.c
drivers/regulator/rk808-regulator.c
drivers/clk/clk-rk808.c
```

RK809 DTS file (refer to the example):

```
arch/arm64/boot/dts/rockchip/px30-evb-ddr4-v10.dts
```

Corresponding macro configuration in menuconfig:

```
CONFIG_MFD_RK808
CONFIG_RTC_RK808
CONFIG_GPIO_RK8XX
CONFIG_REGULATOR_RK808
CONFIG_INPUT_RK8XX_PWRKEY
CONFIG_COMMON_CLK_RK808
```

4.19 Kernel Configuration

RK809 Driver files:

```
drivers/mfd/rk808.c
drivers/input/misc/rk805-pwrkey.c       // Different from the 4.4 kernel
drivers/rtc/rtc-rk808.c
drivers/pinctrl/pinctrl-rk805.c         // Different from the 4.4 kernel
drivers/regulator/rk808-regulator.c
drivers/clk/clk-rk808.c
```

RK809 DTS file (refer to the example):

```
arch/arm64/boot/dts/rockchip/px30-evb-ddr4-v10.dts
```

Corresponding macro configuration in menuconfig:

```
CONFIG_MFD_RK808
CONFIG_RTC_RK808
CONFIG_PINCTRL_RK805
CONFIG_REGULATOR_RK808
CONFIG_INPUT_RK805_PWRKEY
CONFIG_COMMON_CLK_RK808
```

2.2 DTS Configuration

4.4 Kernel DTS Configuration

The DTS configuration includes: I2C bus attachment, main device, RTC, power key, GPIO, regulator, and other sections.

```
&pinctrl {
    pmic {
        pmic_int: pmic_int {
            rockchip,pins =
                <0 RK_PA7 RK_FUNC_GPIO &pcfg_pull_up>;
        };

        soc_slppin_gpio: soc_slppin_gpio {
            rockchip,pins =
                <0 RK_PA4 RK_FUNC_GPIO &pcfg_output_low>;
        };
```

```
        soc_slppin_slp: soc_slppin_slp {
            rockchip,pins =
                <0 RK_PA4 RK_FUNC_1 &pcfg_pull_none>;
        };

        soc_slppin_rst: soc_slppin_rst {
            rockchip,pins =
                <0 RK_PA4 RK_FUNC_2 &pcfg_pull_none>;
        };
    };
};

&i2c1 {
    status = "okay";
    rk809: pmic@20 {
        compatible = "rockchip,rk809";
        reg = <0x20>;
        interrupt-parent = <&gpio0>;
        interrupts = <7 IRQ_TYPE_LEVEL_LOW>;
        pinctrl-names = "default", "pmic-sleep",
                "pmic-power-off", "pmic-reset";
        pinctrl-0 = <&pmic_int>;
        pinctrl-1 = <&soc_slppin_slp>, <&rk817_slppin_slp>;
        pinctrl-2 = <&soc_slppin_gpio>, <&rk817_slppin_pwrdn>;
        pinctrl-3 = <&soc_slppin_rst>, <&rk817_slppin_rst>;
        rockchip,system-power-controller;
        wakeup-source;
        #clock-cells = <1>;
        clock-output-names = "rk808-clkout1", "rk808-clkout2";
        //fb-inner-reg-idxs = <2>;
        /* 1: rst regs (default in codes), 0: rst the pmic */
        pmic-reset-func = <1>;

        vcc1-supply = <&vcc5v0_sys>;
        vcc2-supply = <&vcc5v0_sys>;
        vcc3-supply = <&vcc5v0_sys>;
        vcc4-supply = <&vcc5v0_sys>;
        vcc5-supply = <&vcc3v3_sys>;
        vcc6-supply = <&vcc3v3_sys>;
        vcc7-supply = <&vcc3v3_sys>;
        vcc8-supply = <&vcc3v3_sys>;
        vcc9-supply = <&vcc5v0_sys>;

        pwrkey {
            status = "okay";
        };

        pinctrl_rk8xx: pinctrl_rk8xx {
            gpio-controller;
            #gpio-cells = <2>;

            rk817_slppin_null: rk817_slppin_null {
```

```
            pins = "gpio_slp";
            function = "pin_fun0";
        };

        rk817_slppin_slp: rk817_slppin_slp {
            pins = "gpio_slp";
            function = "pin_fun1";
        };

        rk817_slppin_pwrdn: rk817_slppin_pwrdn {
            pins = "gpio_slp";
            function = "pin_fun2";
        };

        rk817_slppin_rst: rk817_slppin_rst {
            pins = "gpio_slp";
            function = "pin_fun3";
        };
    };

    regulators {
        vdd_logic: DCDC_REG1 {
            regulator-always-on;
            regulator-boot-on;
            regulator-min-microvolt = <950000>;
            regulator-max-microvolt = <1350000>;
            regulator-ramp-delay = <6001>;
            regulator-initial-mode = <0x2>;
            regulator-name = "vdd_logic";
            regulator-state-mem {
                regulator-on-in-suspend;
                regulator-suspend-microvolt = <950000>;
            };
        };

        vdd_arm: DCDC_REG2 {
            regulator-always-on;
            regulator-boot-on;
            regulator-min-microvolt = <950000>;
            regulator-max-microvolt = <1350000>;
            regulator-ramp-delay = <6001>;
            regulator-initial-mode = <0x2>;
            regulator-name = "vdd_arm";
            regulator-state-mem {
                regulator-off-in-suspend;
                regulator-suspend-microvolt = <950000>;
            };
        };
        vcc_ddr: RK809_DCDC3@2 {
            ..................
        };
        ..............................
    };
```

```
        rk809_codec: codec {
            #sound-dai-cells = <0>;
            compatible = "rockchip,rk809-codec", "rockchip,rk817-codec";
            clocks = <&cru SCLK_I2S1_OUT>;
            clock-names = "mclk";
            pinctrl-names = "default";
            pinctrl-0 = <&i2s1_2ch_mclk>;
            hp-volume = <20>;
            spk-volume = <3>;
            status = "okay";
        };
    };
};
```

1. I2C Attachment

The complete RK809 node must be attached under the corresponding I2C node, with `status = "okay";` configured.

2. Main Section

- Do not modify:

```
compatible = "rockchip,rk809";
reg = <0x20>;
rockchip,system-power-controller;
wakeup-source;
gpio-controller;
#gpio-cells = <2>;
```

- Modifiable (according to pinctrl rules)

interrupt-parent: Specifies which GPIO the pmic_int belongs to; interrupts: Specifies the pin index and polarity of pmic_int on the GPIO defined in interrupt-parent; pinctrl-names: Do not modify; fixed as "default"; pinctrl-0: References the pmic_int pin defined in pinctrl;

3. rtc, pwrkey, gpio

If these modules are selected in menuconfig but the corresponding Drivers do not need to be enabled in practice, you can add rtc, pwrkey, and gpio nodes in the DTS and explicitly set their status to status = "disabled". This prevents the Drivers from being enabled, although error logs will appear during boot; these can be ignored. To enable the Drivers, either remove the corresponding nodes or set their status to status = "okay".

4. regulator

- `regulator-compatible`: The name used for matching during Driver registration. Do not modify this value, or the Driver will fail to load.
- `regulator-name`: The power supply name. It is recommended to keep this consistent with the hardware schematic, as the `regulator_get` interface requires a match with this name.
- `regulator-init-microvolt`: The initialization voltage during the U-Boot stage; this is invalid in the kernel stage.
- `regulator-min-microvolt`: The minimum adjustable voltage during runtime.

- `regulator-max-microvolt`: The maximum adjustable voltage during runtime.
- `regulator-initial-mode`: The DCDC operating mode during runtime, typically configured as 1. Values: 1 (force PWM), 2 (auto PWM/PFM).
- `regulator-mode`: The DCDC operating mode during sleep, typically configured as 2. Values: 1 (force PWM), 2 (auto PWM/PFM).
- `regulator-initial-state`: The mode during suspend; must be configured as 3.
- `regulator-boot-on`: When this property is present, the power supply is enabled upon Driver registration.
- `regulator-always-on`: When this property is present, the power supply cannot be disabled during runtime and is enabled upon Driver registration.
- `regulator-ramp-delay`: The DCDC voltage rise time; fixed at 12500.
- `regulator-on-in-suspend`: Maintains power during sleep. To disable the power supply during sleep, change this to `regulator-off-in-suspend`.
- `regulator-suspend-microvolt`: The standby voltage when power is maintained during sleep.

5. Power Off

The RK809 features a unique power-off mechanism that supports direct GPIO control for shutdown. Consequently, the kernel registers a `pm_shutdown_prepare_fn` to perform pre-shutdown preparations, which primarily include disabling the RTC Interrupt and configuring the IOMUX for specific GPIOs. The actual power-off sequence is executed within the PCIE domain, where `pm_power_off` directly toggles the GPIO to shut down the system.

```
static int rk817_shutdown_prepare(struct rk808 *rk808)
{
        int ret;

        /* close rtc int when power off */
        regmap_update_bits(rk808->regmap,
                           RK817_INT_STS_MSK_REG1,
                           (0x3 << 5), (0x3 << 5));
        regmap_update_bits(rk808->regmap,
                           RK817_RTC_INT_REG,
                           (0x3 << 2), (0x0 << 2));

        if (rk808->pins && rk808->pins->p && rk808->pins->power_off) {
                ret = regmap_update_bits(rk808->regmap,
                                         RK817_SYS_CFG(3),
                                         RK817_SLPPIN_FUNC_MSK,
                                         SLPPIN_NULL_FUN);
                if (ret) {
                        pr_err("shutdown: config SLPPIN_NULL_FUN error!\n");
                        return 0;
                }

                ret = regmap_update_bits(rk808->regmap,
                                         RK817_SYS_CFG(3),
                                         RK817_SLPPOL_MSK,
                                         RK817_SLPPOL_H);
                if (ret) {
                        pr_err("shutdown: config RK817_SLPPOL_H error!\n");
                        return 0;
```

```
                }
                ret = pinctrl_select_state(rk808->pins->p,
                                           rk808->pins->power_off);
                if (ret)
                        pr_info("%s:failed to activate pwroff state\n",
                                __func__);
                else
                        return ret;
        }

        /* pmic sleep shutdown function */
        ret = regmap_update_bits(rk808->regmap,
                                 RK817_SYS_CFG(3),
                                 RK817_SLPPIN_FUNC_MSK, SLPPIN_DN_FUN);
        return ret;
}
```

6. Clock Section

If a node needs to reference an RK809 clock, use the following format:

`clocks = <&rk809 1>;` First parameter: &rk809 is fixed and must not be changed. Second parameter: Specifies which RK809 clock to reference; valid values are 0 or 1, where 0 corresponds to rk809-clkout1 and 1 corresponds to rk809-clkout2.

4.19 Kernel DTS Configuration

Refer to Section 4.4 for kernel DTS configuration. Difference: The DTS configuration for the 4.19 kernel no longer requires the gpio subnode, but other modules still reference and use RK809 pins using the format `gpios = <&rk809 0 GPIO_ACTIVE_LOW>;`.

2.3 Function Interfaces

The following interfaces generally meet daily usage requirements, including regulator enable/disable, voltage setting, and voltage retrieval:

1. Obtain a regulator:

`struct regulator *regulator_get(struct device *dev, const char *id)`

Set dev to NULL by default; id corresponds to the regulator-name property in the DTS.

2. Release a regulator:

`void regulator_put(struct regulator *regulator)`

3. Enable a regulator:

`int regulator_enable(struct regulator *regulator)`

4. Disable a regulator:

`int regulator_disable(struct regulator *regulator)`

5. Get regulator voltage:

`int regulator_get_voltage(struct regulator *regulator)`

6. Set regulator voltage

`int regulator_set_voltage(struct regulator *regulator, int min_uV, int max_uV)`

The caller must ensure that the passed parameters satisfy min_uV = max_uV.

7. Example

```
struct regulator *rdev_logic;

rdev_logic = regulator_get(NULL, "vdd_logic");          // Get vdd_logic
regulator_enable(rdev_logic);                           // Enable vdd_logic
regulator_set_voltage(rdev_logic, 1100000, 1100000);    // Set voltage to 1.1V
regulator_disable(rdev_logic);                          // Disable vdd_logic
regulator_put(rdev_logic);                              // Release vdd_logic
```

Note: The 4.4 and 4.19 kernels also provide `devm_`-prefixed regulator interfaces to assist developers in managing allocated resources.

3. Debug

3.1 4.4 Kernel

The command format is identical to that of the 3.10 kernel; only the node path differs. The debug node path for the 4.4 kernel is:

`/sys/rk8xx/rk8xx_dbg`

3.2 4.19 Kernel

Refer to the 4.4 kernel commands.